DATA SCIENCE · MOCK INTERVIEW

Pack C — UrbanEats

Food Delivery Demand Prediction

DOMAIN
Product / Operations

PROBLEM TYPE
Regression + Optimization

DURATION
55 minutes total

⚠️ **INTERVIEWER GUIDE —** Do not share this document with the candidate before or during the session.

Interview Flow

Block	Your Role	Time	Cumulative
Welcome	Ask: *"Tell me about yourself — what brings you here?"*	2 min	2 min
Live Coding	Share problems. Watch screen. Ask comprehension question verbally.	15 min	17 min
Case Setup	Read the scenario aloud. Let them take notes.	3 min	20 min
Case Drive	They drive. Reveal info only when asked. Challenge their choices.	25 min	45 min
Synthesis	*"Can you summarize your recommendation?"*	3 min	48 min
Q&A	They ask you questions. Improvise as a senior DS consultant.	5 min	53 min
Debrief	Score the rubric together. One strength, one area to improve.	10 min	63 min

Case Scenario

Opening Script *(Read Aloud — 2 minutes)*

> *"Our client is **UrbanEats**, a fast-growing food delivery platform based in Mexico City with about 3 million orders per month. They have 800 active riders and partner with 4,000 restaurants across the city.*
>
> *They have a delivery time problem: **25% of their orders take longer than 45 minutes to deliver**, well above their target of 35 minutes average. Customer satisfaction scores have dropped 15 points in the last quarter, and they're losing market share to competitors who deliver faster.*
>
> *The core issue is a supply-demand mismatch — at peak hours in high-demand districts, there simply aren't enough riders. They're throwing money at rider bonuses during peaks, but it's not sustainable. They've asked a consulting team to design a **demand prediction system** that helps them position riders proactively, before the rush hits."*

Information to Reveal — Only When Asked

Rule: Never volunteer information. If they don't ask, they don't get it.

If They Ask About…	What to Say
Current rider allocation	Riders choose where to go. Basic surge pricing (higher pay in busy areas) is reactive, not predictive. By the time surge kicks in, customers are already waiting.
Data available	Two years of order data: timestamps, locations, order values, delivery times, weather. Rider data: home district, vehicle type, working hours, ratings. Restaurant data: location, cuisine type, prep time estimates.
Demand patterns	**Show Exhibit 1.** Lunch peak 12–1 PM, dinner peak 7–9 PM. Weekends significantly higher, especially Saturday evenings. Friday nights are also big.
Supply-demand gap	**Show Exhibit 2.** Centro and Roma/Condesa have the biggest gaps — demand exceeds rider supply by 30%+. Peripheral districts like Tlalpan are more balanced but have longer delivery distances.
Weather impact	Rain increases demand by 20% (people don't go out) but decreases rider availability by 30% (riders don't want to ride in rain). Heavy rain is a double hit.

Delivery time breakdown	Avg 38 min: 5 min rider acceptance, 8 min travel to restaurant, 12 min restaurant prep, 13 min delivery. Biggest variable: rider travel to restaurant — that's where positioning helps.
Competitor landscape	Rappi has a similar model. DiDi Food started using demand prediction and pre-positioning — their delivery times dropped 8 minutes last quarter.

Exhibits *(Screen-share only when asked)*

EXHIBIT 1 — Show when asked about demand patterns

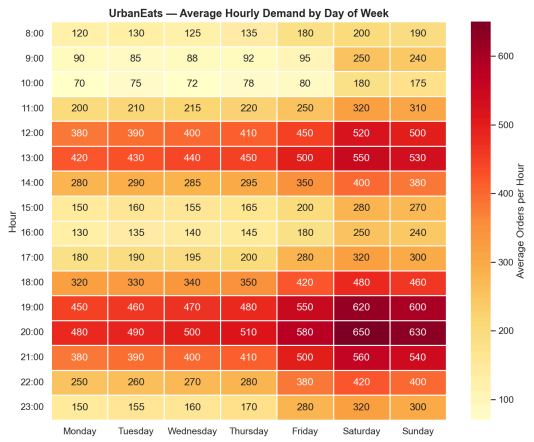
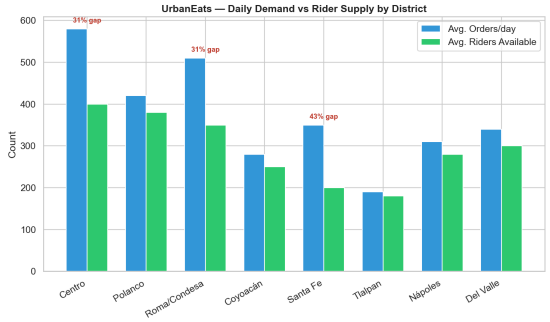


EXHIBIT 2 — Show when asked about supply-demand gap



Challenges to Pose *(Pick 2–3 during Structure phase)*

On forecast accuracy requirements

"You want to predict demand 2 hours ahead. How accurate does the forecast need to be for rider pre-positioning to actually work? What's the cost of over-prediction vs under-prediction?"

On the incentive problem

"Riders are independent contractors — you can't TELL them where to go. How does your prediction system translate into actual rider behavior?"

On weather uncertainty

"You mentioned weather as a feature. But weather forecasts are unreliable more than a few hours out in Mexico City. How do you handle that uncertainty in your model?"

On root cause

> *"Centro has the biggest demand-supply gap. Is the solution more riders in Centro, or is it something else entirely?"*

The Twist *(Drop around minute 18)*

DROP THIS AT 18:00

"One more thing — in exactly two weeks, Vive Latino (a massive music festival) is happening in Mexico City. Last year's edition wasn't tracked because UrbanEats was smaller then. They expect 80,000 attendees per day for 3 days, concentrated near the Foro Sol area. There's no historical data for an event like this. The CEO wants your system to handle it. What do you do?"

What a strong answer looks like: Special event handling — historical data won't help for unprecedented events. Options: (1) Use proxy data from similar events (concerts, sports) to estimate demand multiplier. (2) Pre-position riders near Foro Sol proactively based on event schedule (when sets end = order surge). (3) Partnerships with nearby restaurants for increased capacity. (4) Extended surge pricing window. (5) Proactively communicate expected delays to customers. Key insight: the model needs an **operator override** mechanism where expert knowledge can be injected manually.

What a Strong Candidate Covers

1. **Reframes clearly:** *"How do we predict demand 1–2 hours ahead at the district level so UrbanEats can pre-position riders and reduce >45 min deliveries from 25% to under 10%?"*
2. **Separates the two sub-problems:** Demand prediction (regression) + rider optimization (allocation). Strong candidates name both explicitly.
3. **Features for demand model:** Hour of day, day of week, weather, district, holidays/events, historical trends, promotional campaigns, restaurant density.
4. **Model + optimization layer:** Gradient boosting or time series model for demand. Separate optimization layer for rider allocation. Not one magic model.
5. **Addresses the incentive problem:** Prediction alone doesn't move riders — needs incentive design (predictive surge pricing, guaranteed minimums for pre-positioned riders).
6. **Rain paradox:** Demand up + supply down = worst case. Model needs to account for both supply and demand simultaneously.
7. **Event handling:** Pragmatic approach — proxy data, expert overrides, proactive customer communication.

8 **Quantifies:** 3M orders × 25% delayed = 750K bad experiences/month. Reducing to 10% = 450K fewer. At 5% churn probability × $6,000 MXN LTV = **$135M MXN annual value.**

Timing Guide

Phase	Target Time	Notes
Read scenario aloud	0:00 – 2:00	
Reframe + their questions	2:00 – 7:00	Show exhibits when relevant
Their structure	7:00 – 20:00	Challenge at 12:00 and 16:00
Drop the twist	18:00	Festival — no historical data
Synthesis	22:00 – 25:00	*"Wrap up with a recommendation?"*

Coding Problems

Problem 1 — Delivery Performance by District *(7 minutes)*

Two CSV files:

- `orders.csv` — `order_id, timestamp, hour, district, order_value_mxn, items, is_premium_customer, delivery_time_min, weather`
- `riders.csv` — `rider_id, home_district, avg_deliveries_per_day, avg_rating, vehicle_type, active_hours_per_week, months_active`

Note: Some values may be missing.

Write a Python function returning a DataFrame with one row per **district**, containing:

```
1. district
2. total_orders — Total orders
3. avg_delivery_time — Avg delivery time in minutes, nulls excluded (1 decimal)
4. pct_delayed — % of orders with delivery time > 45 min (1 decimal)
5. avg_order_value — Avg order value in MXN, nulls excluded (2 decimal)
6. rider_count — Number of riders whose home_district matches (from riders.csv)
7. orders_per_rider — total_orders / rider_count (1 decimal)

→ Sort by pct_delayed descending. Aggregate orders and riders separately, then
merge.
```

Problem 2 — Weather & Peak Hour Impact *(7 minutes)*

Using the same `orders.csv`. Define peak hours as 12–13 (lunch) and 19–21 (dinner). All other hours are off-peak.

Write a function returning one row per **weather condition × peak/off-peak**, containing:

```
• weather
• is_peak — True/False
• order_count
• avg_delivery_time (1 decimal)
• pct_delayed — % > 45 min (1 decimal)
• avg_order_value (2 decimal)

→ Sort by avg_delivery_time descending.
Bonus: What's the worst combination of weather + peak? How much worse than the
best case?
```

Code Comprehension — Conversational *(no coding required)*

```python
def optimize_rider_allocation(demand_forecast, rider_pool, max_distance_km=5):
    allocation = {}
    available = rider_pool.copy()

    demand_sorted = demand_forecast.sort_values('predicted_orders', ascending=False)

    for _, district in demand_sorted.iterrows():
        district_name = district['district']
        needed = int(np.ceil(district['predicted_orders'] / 8))

        nearby = available[available['home_district'] == district_name]

        if len(nearby) >= needed:
            assigned = nearby.head(needed)
        else:
            assigned = nearby.copy()
            remaining_need = needed - len(nearby)
            other = available[available['home_district'] != district_name]
            assigned = pd.concat([assigned, other.head(remaining_need)])

        allocation[district_name] = assigned['rider_id'].tolist()
        available = available[~available['rider_id'].isin(assigned['rider_id'])]

    return allocation
```

Ask these three questions verbally:

1. **"Walk me through the logic of this function."**
   - Expected: A greedy rider allocation algorithm. Sorts districts by predicted demand (highest first), calculates needed riders (orders ÷ 8), assigns local riders first, then pulls from other districts if needed. Once assigned, riders are removed from the available pool.
2. **"What's the problem with this greedy approach?"**
   - Expected: High-demand districts get first pick of local riders — lower-demand districts may get riders from far away even if a balanced allocation would reduce total travel time. Also: `max_distance_km` is defined but **never used**.
3. **"How would you improve this?"**
   - Expected: Linear programming to minimize total rider travel time while meeting demand. Consider vehicle type (bicycle vs motorcycle). Account for the unused `max_distance_km` to filter eligible riders from other districts.

Interviewer Rules

→ **Screen sharing required** for coding. They can look up syntax — no AI tools.

→ **Never volunteer information.** If they don't ask, they don't get it.

→ **Challenge model choices.** "How does your prediction translate into actual rider movement?" "What happens during a rain event?"

→ **Drop the festival twist** at 18 minutes.

→ **Use the scoring rubric** — fill it out during or right after.

→ **Keep it conversational.** You're a friendly colleague, not an examiner.

Make it real. The best gift you can give your friend is an honest, challenging mock. Push back. Don't go easy. Good luck! 🏔️